

Azure Cloud Fundamentals and Data Pipeline Implementation using ADF

This assignment is based on creating a simple end-to-end data pipeline using Azure Blob Storage and Azure Data Factory. I used the Superstore CSV dataset as the source file and copied it to a new destination location using an ADF pipeline.

Objective

The main objective of this assignment is to understand the basic Azure services used in a data pipeline. In this project, I created the required Azure resources, uploaded the Superstore dataset to Blob Storage, and used Azure Data Factory to copy the data from the source container to a destination container.

I also used the Get Metadata activity to check the source file information before the copy process.

Project Architecture

The pipeline flow used in this assignment is:

Superstore CSV → Azure Blob Storage → Azure Data Factory → Destination Blob Storage

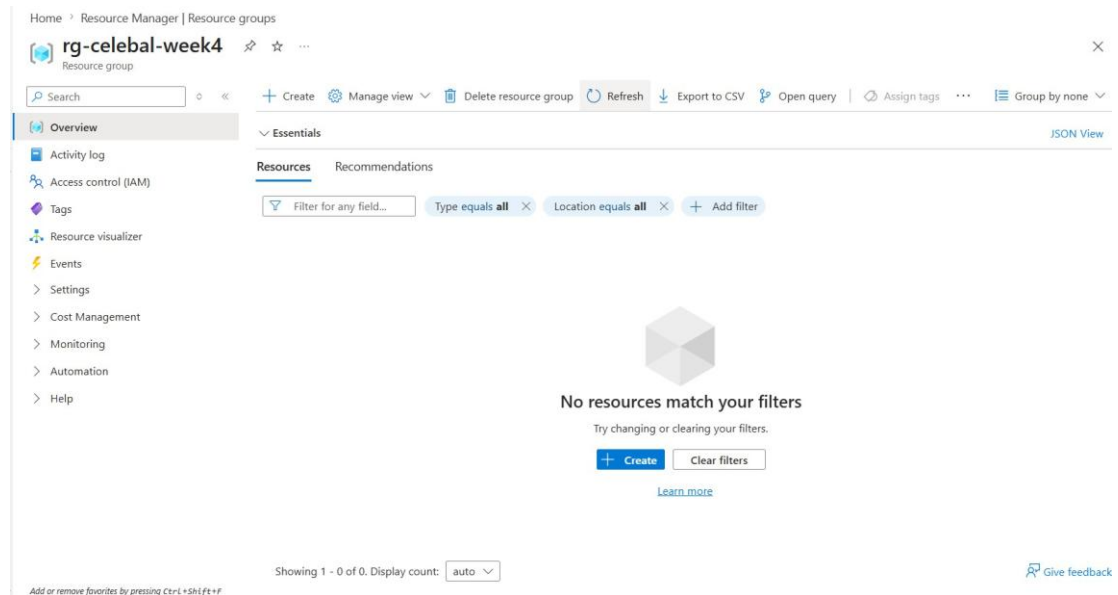
The source CSV file is stored in a Blob container. Azure Data Factory connects to the storage account using a Linked Service. The pipeline first uses Get Metadata to check the source file and then uses Copy Data to move the file to the destination location.

Task 1: Create a Resource Group

I started by creating a Resource Group for the Week 4 assignment. I kept all the Azure resources for this project in the same Resource Group so that they are easier to organize and manage.

Resource Group Name: rg-celebal-week4

Region: Central India

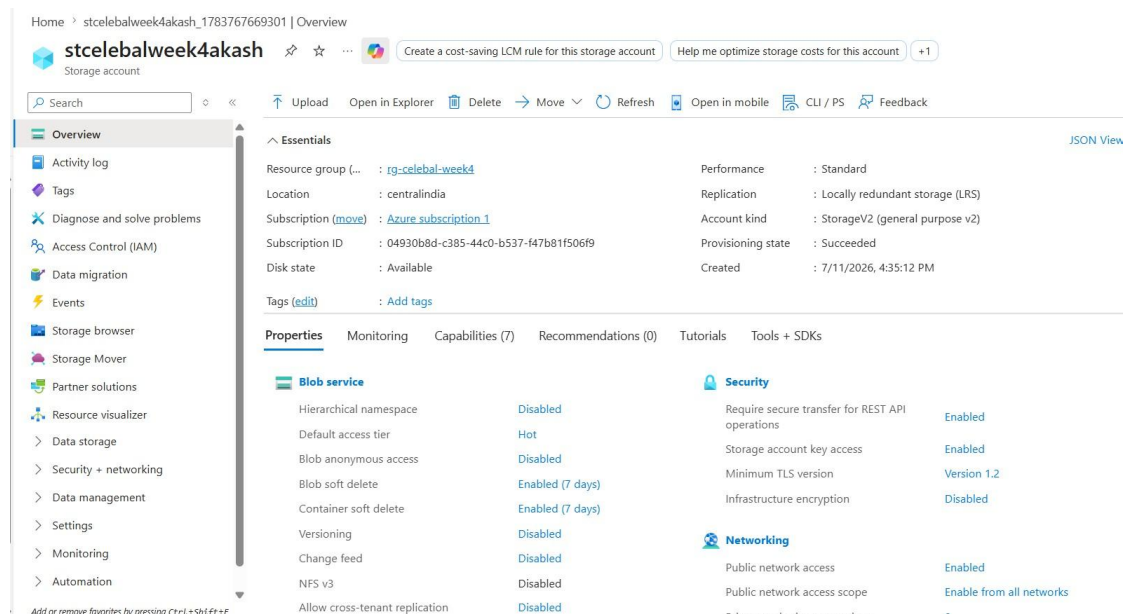


I created a separate Resource Group for this assignment. This keeps the Storage Account and Azure Data Factory resources related to the same project in one place.

Task 2: Storage Setup

In this task, I created an Azure Storage Account to store the dataset used in the pipeline. Azure Storage provides cloud-based storage services, and Blob Storage is commonly used for storing unstructured data such as CSV files.

Storage Account Name: stcelebalweek4akash

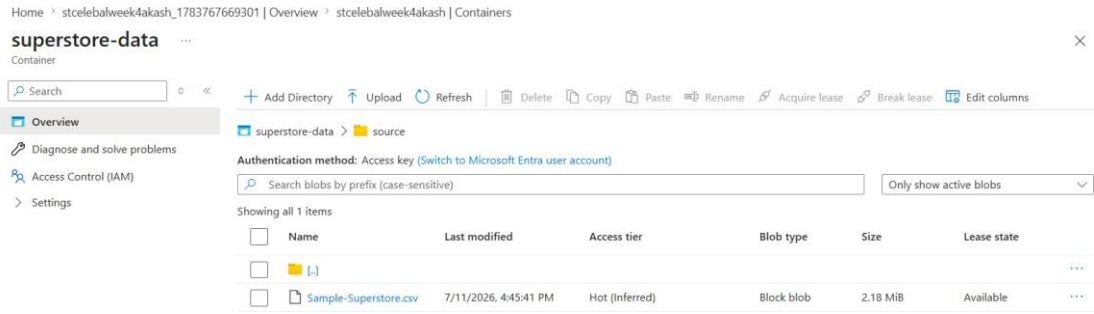


After creating the Storage Account, I created a Blob Container named `superstore-data`. Inside the container, I created a `source` folder and uploaded the `Sample-Superstore.csv` file.

Container Name: superstore-data

Source Folder: source

Source File: Sample-Superstore.csv

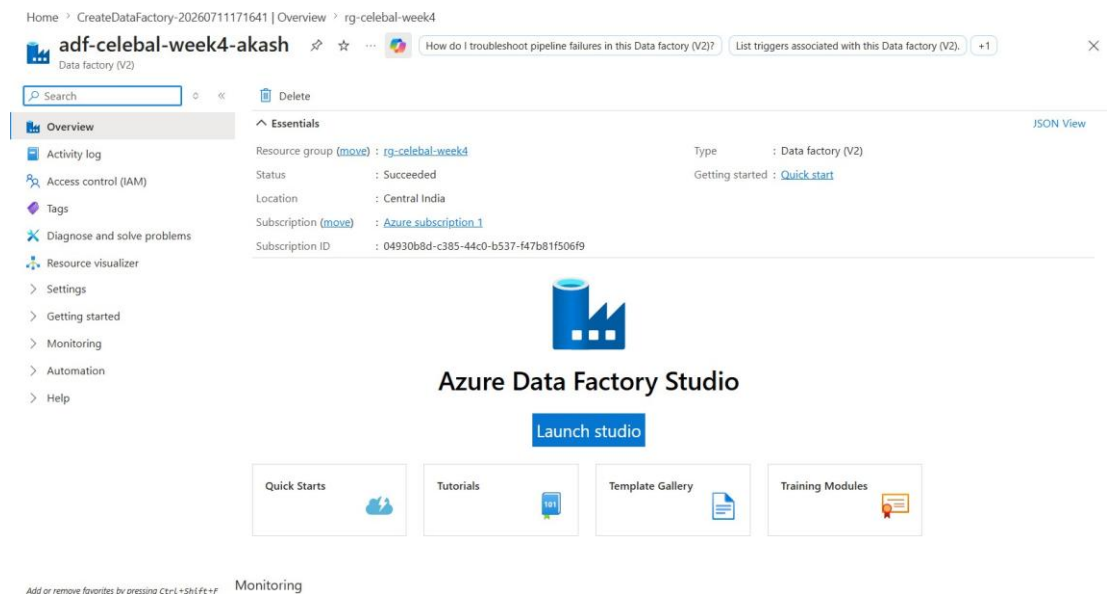


The CSV file was uploaded successfully and is used as the source data for the Azure Data Factory pipeline. Using separate source and destination locations also makes the data movement process easier to understand and manage.

Task 3: Azure Data Factory Basics

In this task, I created an Azure Data Factory to build and manage the data pipeline. Azure Data Factory (ADF) is a cloud-based data integration service that can be used to connect different data sources, move data, and create data processing workflows.

Data Factory Name: adf-celebal-week4-akash



ADF Studio

I explored the main sections of Azure Data Factory Studio:

- **Author:** Used to create pipelines, datasets, and activities.
- **Monitor:** Used to check pipeline runs and activity execution status.
- **Manage:** Used to configure linked services, integration runtimes, and other connections.

Linked Service

I created a Linked Service named `LS_AzureBlobStorage` to connect Azure Data Factory with the Azure Blob Storage account. A Linked Service stores the connection information required by ADF to access an external data source or service.

I also tested the connection to verify that Azure Data Factory could successfully connect to the Storage Account.

New linked service

 Azure Blob Storage [Learn more](#) 

Name *

LS_AzureBlobStorage

Description

Connection to Azure Blob Storage for Superstore data



Connect via integration runtime * 

 AutoResolveIntegrationRuntime

Authentication type

Account key

Connection string

Azure Key Vault

Account selection method 

☒ From Azure subscription ☐ Enter manually

Azure subscription 

Azure subscription 1 (04930b8d-c385-44c0-b537-f47b81f506f9)

Storage account name *

stcelebalweek4akash



Additional connection properties

 Connection successful

Create

Back



Test connection

Cancel

Linked services

Linked service defines the connection information to a data store or compute. [Learn more](#) 

 New

 Filter by name

Annotations : Any

Showing 1 - 1 of 1 items

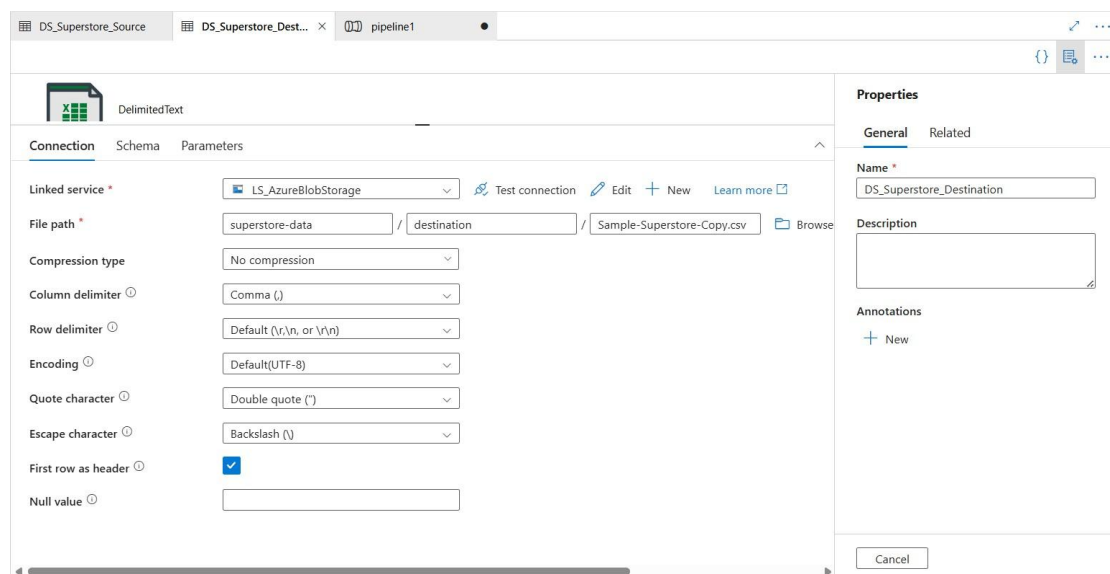
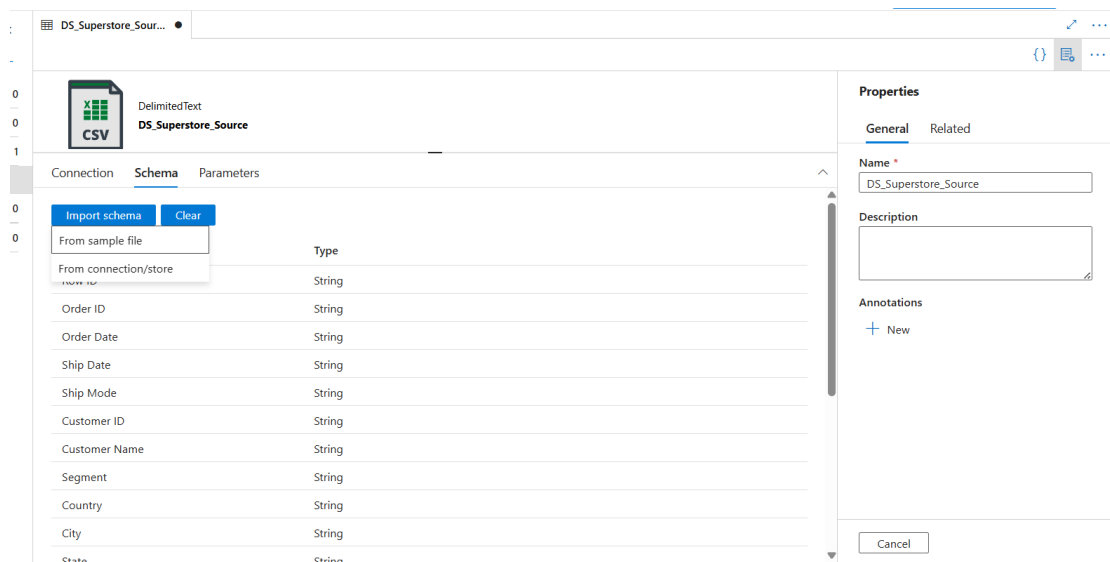
Name 	Type 	Related 	Annotations 
 LS_AzureBlobStorage	Azure Blob Storage	0	

Source and Destination Datasets

After creating the Linked Service, I created two datasets:

- `DS_Superstore_Source` for reading the source CSV file.
- `DS_Superstore_Destination` for writing the copied data to the destination location.

A dataset represents the structure and location of the data that an activity reads from or writes to.

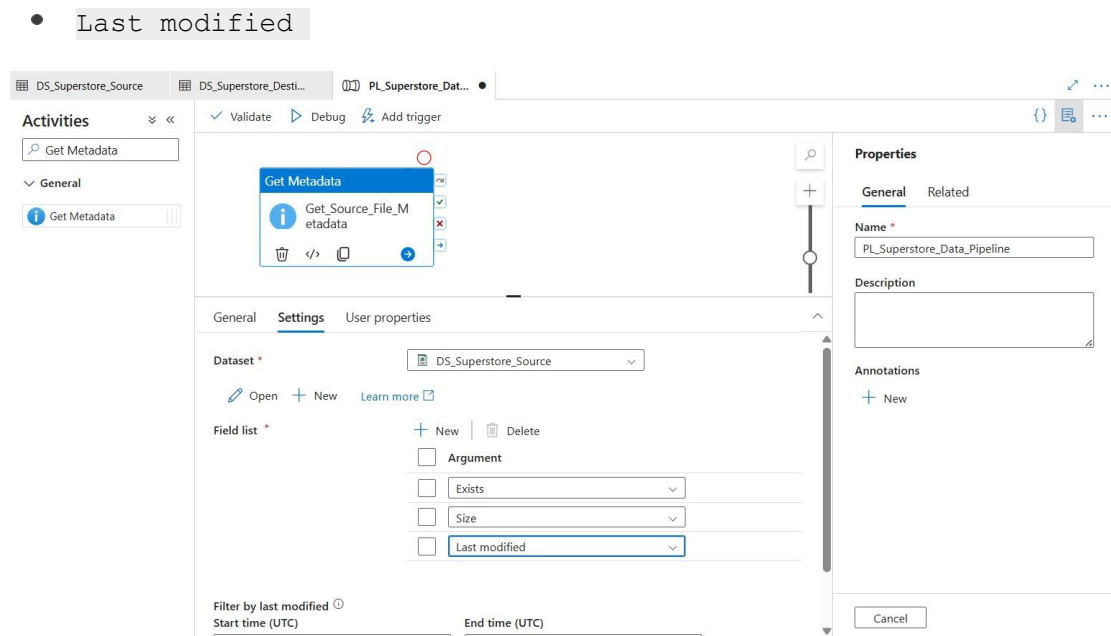


Get Metadata Activity

I added a Get Metadata activity named `Get_Source_File_Metadata` to the pipeline. This activity was configured with the source dataset to retrieve information about the source file before the Copy Data activity runs.

The metadata fields selected were:

- `Exists`
- `Size`



The Get Metadata activity helps validate the source file and provides useful information about it before the data is copied. This makes the pipeline flow more reliable because the source file is checked before the next activity is executed.

Task 4: Pipeline Development

In this task, I created a pipeline named `PL_Superstore_Data_Pipeline` to move the Superstore CSV data from the source location to the destination location.

The pipeline contains two main activities:

1. `Get_Source_File_Metadata` - retrieves metadata about the source CSV file.
2. `Copy_Superstore_Data` - copies the data from the source dataset to the destination dataset.

The activities are connected using a **Success dependency**, shown by the green connection. This means that the Copy Data activity runs only after the Get Metadata activity completes successfully.

For the Copy Data activity:

- **Source Dataset:** `DS_Superstore_Source`
- **Destination Dataset:** `DS_Superstore_Destination`

The source and destination columns were mapped so that the structure of the CSV data remains consistent during the copy operation.

The pipeline flow follows this sequence:

```

graph LR
    A[Get Metadata: Get_Source_File_Metadata] --> B[Copy data: Copy_Superstore_Data]
  
```

Source	Type	Destination
☐ Row ID	abc String	Row ID
☐ Order ID	abc String	Order ID
☐ Order Date	abc String	Order Date
☐ Ship Date	abc String	Ship Date
☐ Ship Mode	abc String	Ship Mode
☐ Customer ID	abc String	Customer ID
☐ Customer Name	abc String	Customer Name
☐ Segment	abc String	Segment

The pipeline flow follows this sequence:

Get Metadata → Copy Data

This design separates the metadata check from the actual data movement. If the metadata activity fails, the Copy Data activity will not continue through the success path. A `ForEach` activity was not required in this pipeline because only one CSV file was processed.

Task 5: Pipeline Execution

Before running the pipeline, I validated it to check for configuration errors. The validation completed successfully with no errors.

Pipeline validation output

Your pipeline has been validated.
No errors were found.

After validation, I executed the pipeline using the Debug option. The pipeline ran in the following sequence:

Get_Source_File_Metadata → Copy_Superstore_Data

Both activities completed successfully, and the overall pipeline status was Succeeded .

The screenshot shows the Azure Data Factory console with a pipeline named 'PL_Superstore_Data' in a 'Debug' state. The pipeline consists of two activities: 'Get Metadata' (Get_Source_File_Metadata) and 'Copy data' (Copy_Superstore_Data). The 'Output' tab displays the following table:

Activity name	Activity st...	Activit...	Run start	Duration
Copy_Superstore_Data	✓ Succeeded	Copy data	7/11/2026, 9:42:15 PM	15s
Get_Source_File_Metadata	✓ Succeeded	Get Metadata	7/11/2026, 9:41:58 PM	16s

After the successful pipeline execution, I checked the destination folder in the Blob Storage container. The copied file Sample-Superstore-Copy.csv was successfully created in the destination location.

The screenshot shows the Azure Storage Explorer interface for a container named 'superstore-data'. The 'destination' folder is selected, and a table of blobs is displayed:

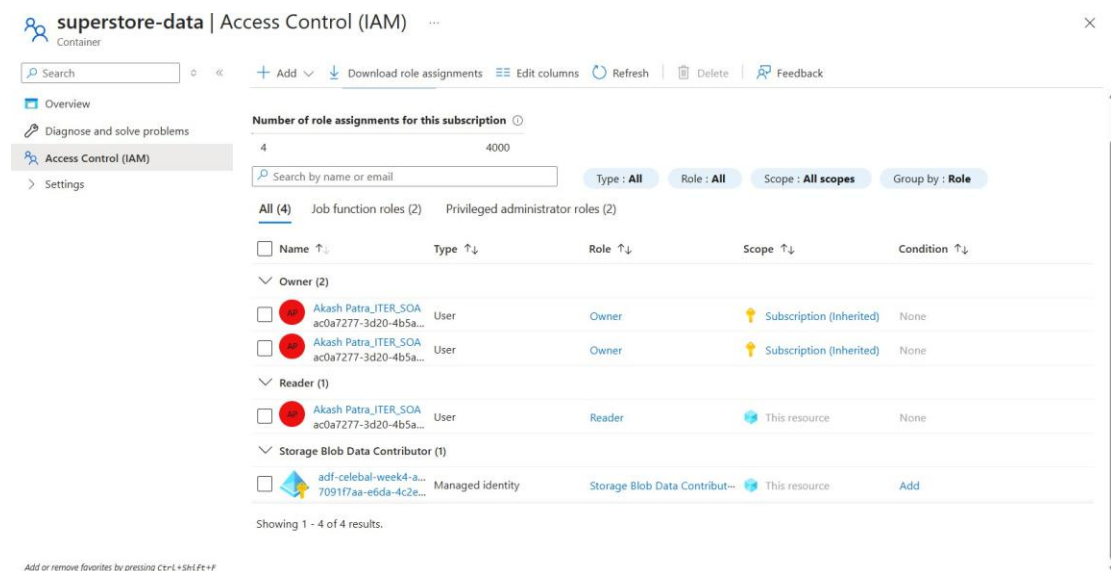
Name	Last modified	Access tier	Blob type	Size	Lease state
Sample-Superstore-Copy.csv	7/11/2026, 9:42:26 PM	Hot (Inferred)	Block blob	2.58 MiB	Available

This confirms that the complete data movement process worked successfully. The source file was checked using the Get Metadata activity, the data was copied using the Copy Data activity, and the output file was successfully created in the destination location.

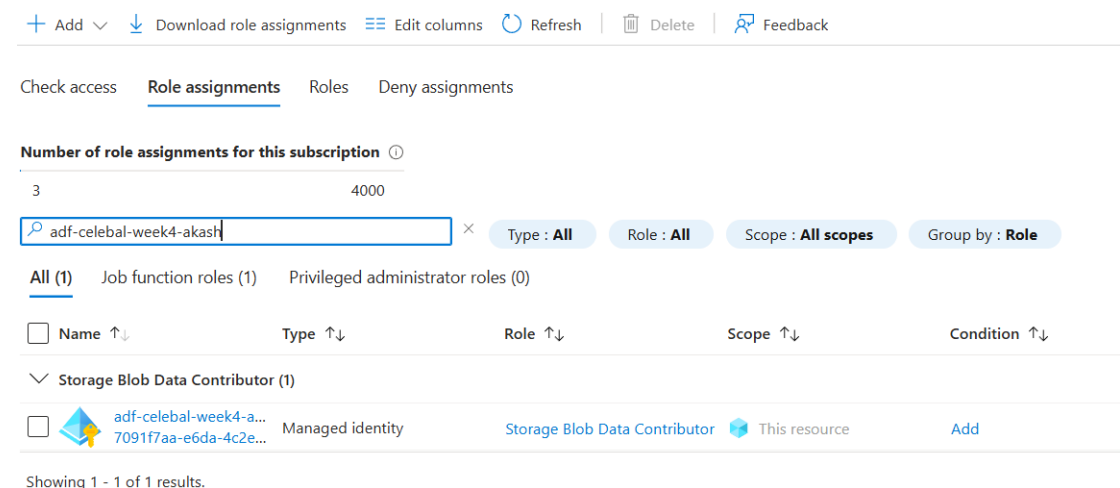
Task 6: IAM Roles and Access Control

In this task, I checked and configured access permissions using Azure Role-Based Access Control (RBAC). RBAC is used to control who or what can access Azure resources and what actions they are allowed to perform.

The role assignments showed that my user account had the **Reader** role for viewing the resource. My account also had inherited **Owner** access from the subscription level.



To provide Azure Data Factory access to the Blob Storage data, I assigned the **Storage Blob Data Contributor** role to the managed identity of **adf-celebal-week4-akash**.



The **Storage Blob Data Contributor** role allows the Data Factory managed identity to read, write, and manage blob data in the assigned storage scope. This provides controlled access between Azure Data Factory and Blob Storage using Azure RBAC.

The assignment also required checking Reader and Contributor-level access. In my current Azure environment, the **Reader** role was already assigned and my user

account had inherited `Owner` access, which provides broader resource-management permissions. For the ADF-to-Storage connection, the specific `Storage Blob Data Contributor` role was assigned to the Data Factory managed identity.

Mini Project: End-to-End Blob Storage to ADF Data Pipeline

Problem Statement

The goal of this mini project was to build a complete data pipeline that reads a CSV file from Azure Blob Storage, checks the source file metadata, and copies the data to a new destination location using Azure Data Factory.

Pipeline Architecture

The complete data flow used in this project is:

```
Sample-Superstore.csv → Azure Blob Storage (Source) → Get  
Metadata → Copy Data → Azure Blob Storage (Destination)
```

Source

The source file used in this project was:

- **File:** `Sample-Superstore.csv`
- **Container:** `superstore-data`
- **Source Directory:** `source`

ADF Components Used

The following Azure Data Factory components were used:

- **Linked Service:** `LS_AzureBlobStorage`
- **Source Dataset:** `DS_Superstore_Source`
- **Destination Dataset:** `DS_Superstore_Destination`
- **Pipeline:** `PL_Superstore_Data_Pipeline`
- **Get Metadata Activity:** `Get_Source_File_Metadata`
- **Copy Data Activity:** `Copy_Superstore_Data`

Process

The pipeline first uses the Get Metadata activity to check information about the source CSV file, including its existence, size, and last modified time.

After the metadata activity completes successfully, the Copy Data activity reads the data from the source dataset and writes it to the destination dataset.

The final pipeline flow is:

```
Get_Source_File_Metadata → Copy_Superstore_Data
```

Destination

The copied file was created at the following location:

- **Container:** superstore-data
- **Destination Directory:** destination
- **Output File:** Sample-Superstore-Copy.csv

Final Result

The pipeline executed successfully. Both the Get Metadata and Copy Data activities completed with a `Succeeded` status.

The source CSV file was successfully copied from the `source` directory to the `destination` directory, and the output file was verified in Azure Blob Storage.

Key Learnings

From this assignment, I learned how different Azure services can work together in a simple data pipeline. I understood how a Storage Account and Blob Container are used to store source and destination files, and how Azure Data Factory connects to Blob Storage using a Linked Service.

I also learned the difference between a Linked Service and a Dataset. The Linked Service creates the connection to the storage service, while the Dataset points to the actual data location and format.

The Get Metadata activity helped me understand how file information can be checked before performing the next operation. I also learned how activities can be connected using a success dependency so that the next activity runs only after the previous activity completes successfully.

Possible Improvements

This pipeline currently processes a single CSV file. If multiple files need to be processed in the future, a `ForEach` activity can be added to process each file dynamically.

The pipeline can also be improved by adding validation conditions after the Get Metadata activity. For example, an `If Condition` activity can be used to check whether the source file exists before starting the Copy Data activity.

For a larger project, pipeline parameters can also be used to make the source and destination file paths dynamic instead of using fixed file names.

Conclusion

This assignment helped me understand the basic flow of an end-to-end Azure data pipeline. I successfully created the required Azure resources, connected Azure Data Factory with Blob Storage, checked the source file metadata, copied the CSV data to a new destination location, and verified the successful pipeline execution.